Heart Disease Prediction — End-to-End MLOps Project Report

Course: Machine Learning Operations (MLOps) AIMLCZG523 — Assignment 01
Problem: Predict the risk of heart disease from patient health data and deploy the solution as a cloud-ready, monitored API.
GitHub Repository: https://github.com/agartanu-git/heart-disease-mlops

Python 3.9 | scikit-learn 1.5.1 | MLflow | FastAPI | Docker | Kubernetes + Helm | GitHub Actions | Prometheus + Grafana

---

1. Setup & Installation

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# End-to-end pipeline
bash run_pipeline.sh          # download -> EDA -> train
uvicorn src.api:app --port 8000   # serve
```

Conda alternative: `conda env create -f environment.yml`.

All scripts execute from a clean checkout; paths are resolved relative to the project root (`src/config.py`), and dependencies are pinned in `requirements.txt`.

2. Data Acquisition & EDA

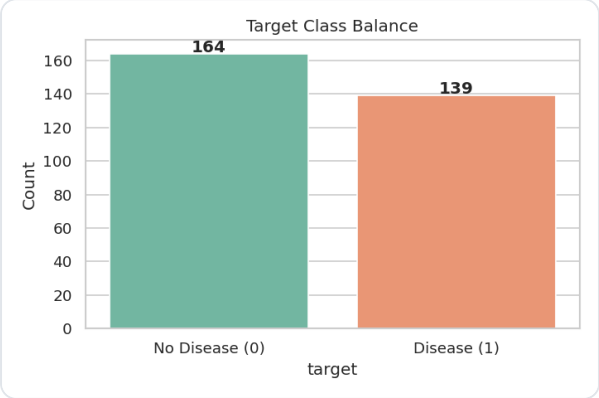
Dataset: UCI Heart Disease (processed Cleveland) — 303 records, 13 features, 1 target. Downloaded automatically by `scripts/download_data.py` (honours HTTP/HTTPS proxy, with a mirror fallback).

Cleaning (`src/data_preprocessing.py`)

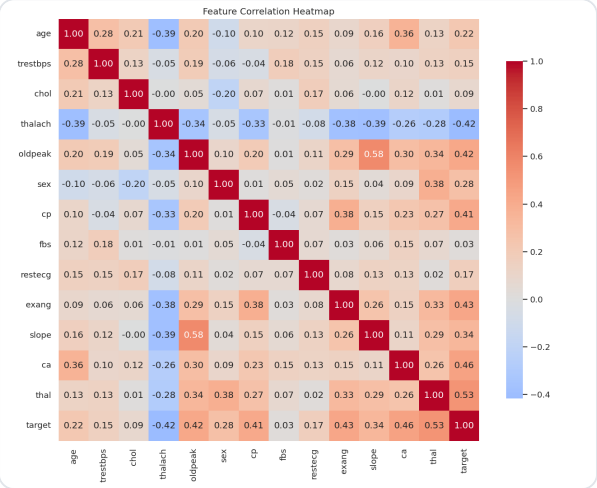
- `?` markers coerced to `NaN`; duplicates dropped.

- Multi-class severity target (0–4) binarised to **0 = no disease** / **1 = disease**.
- All features coerced to numeric.

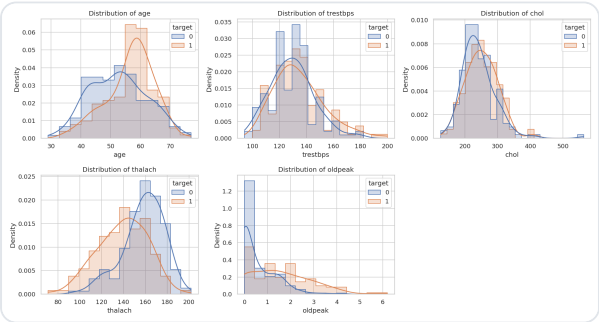
Class balance: 164 negative vs 139 positive — fairly balanced, so accuracy is a reasonable headline metric alongside ROC-AUC.



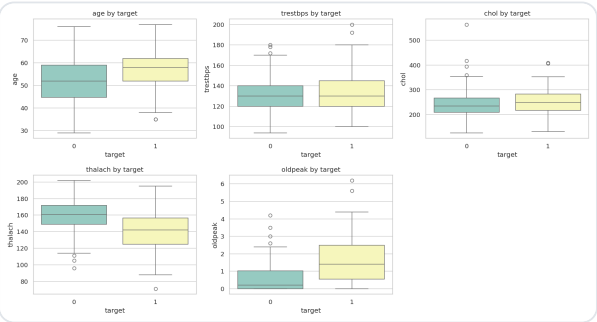
Target class balance (0 = no disease, 1 = disease).



Feature correlation heatmap.



Numeric feature distributions by target class.



Numeric features by target (outlier & spread view).

Key EDA findings: `cp` (chest pain type), `thalach` (max heart rate, negatively correlated with disease), `oldpeak`, `ca`, and `thal` show the strongest association with the target.

3. Feature Engineering & Model Development

Preprocessing pipeline (fitted only on training data to avoid leakage)

- Numeric (`age, trestbps, chol, thalach, oldpeak`): median impute → standard scale.
- Categorical (`sex, cp, fbs, restecg, exang, slope, ca, thal`): most-frequent impute → one-hot encode (`handle_unknown="ignore"`).

Models trained (each inside a full `Pipeline`, tuned with `GridSearchCV`, 5-fold stratified CV, scored on ROC-AUC):

1. Logistic Regression — grid over `C`, `penalty`, `solver`.
2. Random Forest — grid over `n_estimators`, `max_depth`, `min_samples_split`.

Results (held-out 20% test set)

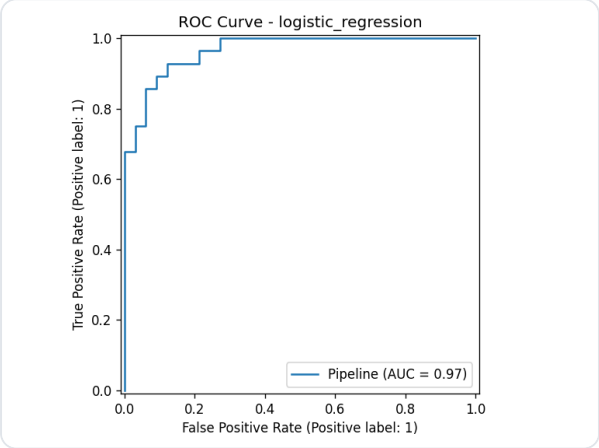
Model	CV ROC-AUC	Test Acc	Precision	Recall	F1	Test ROC-AUC
Logistic Regression ✓	0.899	**0.885**	0.839	0.929	0.881	**0.966**
Random Forest	0.903	0.836	0.821	0.821	0.821	0.944

Model selection: Logistic Regression was selected as the production model (highest test ROC-AUC = 0.966 and best recall = 0.93, which matters for a screening task where missing a positive case is costly). Best hyper-parameters: `C=0.1, penalty=l2, solver=liblinear`.

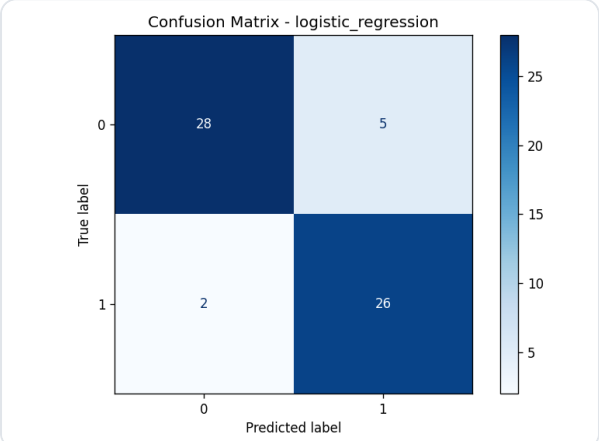
Screenshot S1 — End-to-end pipeline run: data download (303 rows), EDA class balance, and both models' cross-validated & test metrics, selecting Logistic Regression (ROC-AUC = 0.9665) as the best model.

```
(.venv) [root@f6-ftc28-26s1-mgmt heart-disease-mlops]#
(.venv) [root@f6-ftc28-26s1-mgmt heart-disease-mlops]# python scripts/download_data.py && python -m src.eda && python -m src.t
rain 2>&1 | grep -Ev "WARNING|UserWarning|warnings.warn|GIT_PYTHON"
Downloading from https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/processed.cleveland.data ...
Saved 303 rows x 14 cols -> /root/heart-disease-mlops/data/raw/heart_disease.csv
{
  "0": 164,
  "1": 139
}
EDA figures written to /root/heart-disease-mlops/reports/figures
[logistic_regression] best_params={'model__C': 0.1, 'model__penalty': 'l2', 'model__solver': 'liblinear'}
[logistic_regression] cv_roc_auc=0.8989 test={'accuracy': 0.8852459016393442, 'precision': 0.8387096774193549, 'recall': 0.928
5714285714286, 'f1': 0.8813559322033898, 'roc_auc': 0.9664502164502164}
[random_forest] best_params={'model__max_depth': 4, 'model__min_samples_split': 2, 'model__n_estimators': 100}
[random_forest] cv_roc_auc=0.9029 test={'accuracy': 0.8360655737704918, 'precision': 0.8214285714285714, 'recall': 0.821428571
4285714, 'f1': 0.8214285714285714, 'roc_auc': 0.9437229437229436}

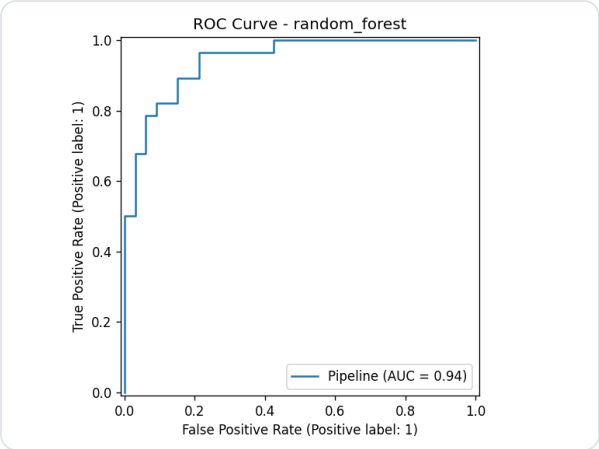
Best model: logistic_regression (ROC-AUC=0.9665) -> /root/heart-disease-mlops/models/model.joblib
(.venv) [root@f6-ftc28-26s1-mgmt heart-disease-mlops]#
```



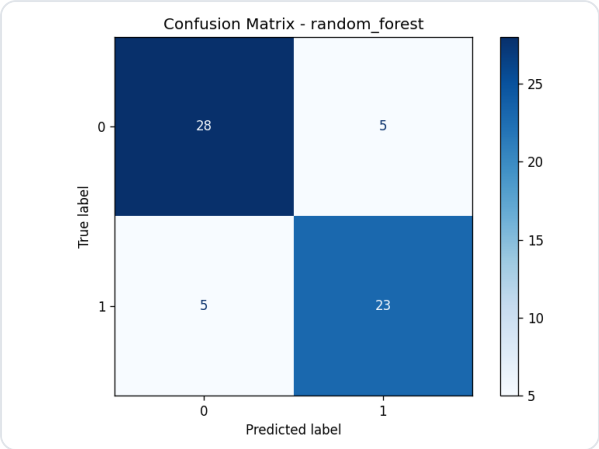
ROC curve — Logistic Regression (selected model).



Confusion matrix — Logistic Regression.



ROC curve — Random Forest.



Confusion matrix — Random Forest.

4. Experiment Tracking (MLflow)

`src/train.py` logs **every run** to MLflow (`./mlruns`):

- Parameters: model type + tuned hyper-parameters.
- Metrics: `cv_roc_auc` and all test metrics.
- Artifacts: ROC curve, confusion matrix, and the serialized sklearn model.

```
mlflow ui --backend-store-uri ./mlruns   # http://localhost:5000
```

Experiment name: `heart-disease-classification`.

5. Model Packaging & Reproducibility

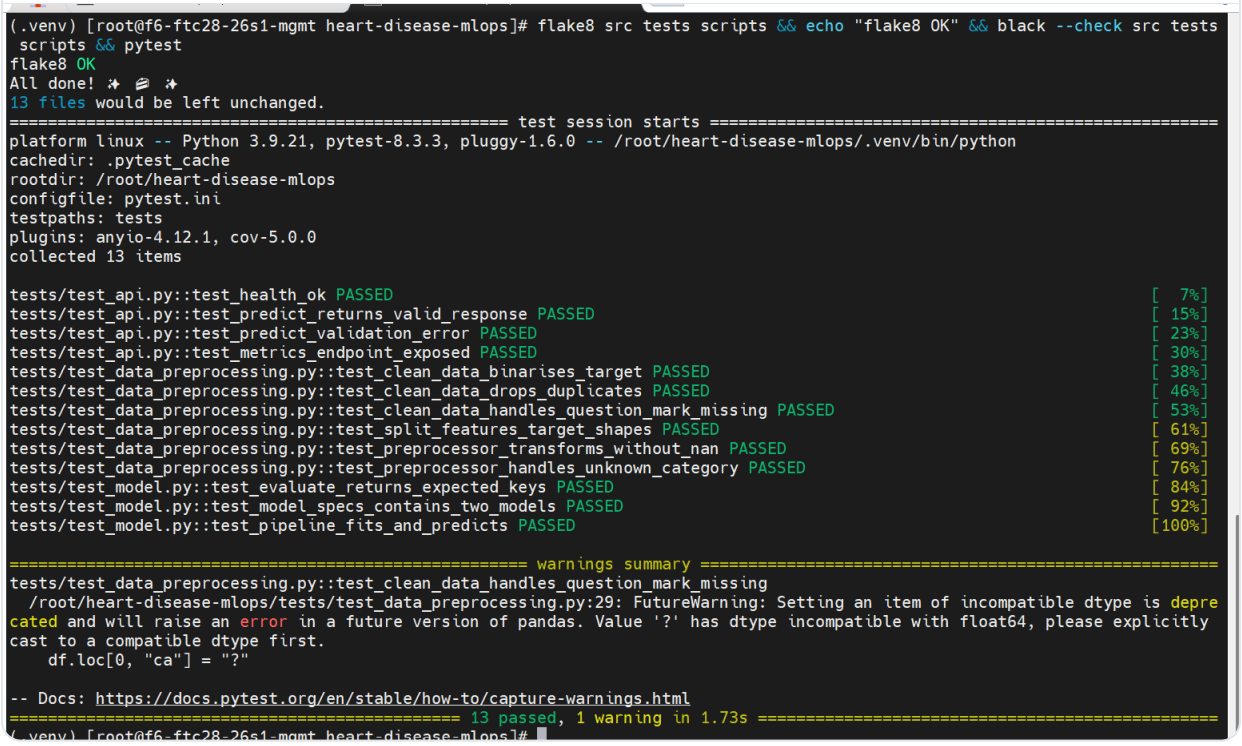
- Final pipeline persisted with `joblib` → `models/model.joblib` (preprocessing + estimator in one object, so inference uses the exact training transforms).
- Metrics summary → `models/metrics.json`.
- Deterministic train/test split (`RANDOM_STATE=42`) saved to `data/processed/{train,test}.csv`.
- Pinned `requirements.txt` + `environment.yml`.

6. CI/CD & Automated Testing

Tests (`tests/`, pytest) — 13 tests covering:

- Data cleaning (binarisation, dedup, missing-value handling).
- Preprocessing (no NaNs after transform, unknown-category robustness).
- Model (metric computation, pipeline fit/predict).
- API integration (`/health`, `/predict`, validation 422, `/metrics`).

Screenshot S2 — Quality gate passing locally: `flake8` clean, `black --check` reports no changes, and all **13 pytest tests PASSED**.

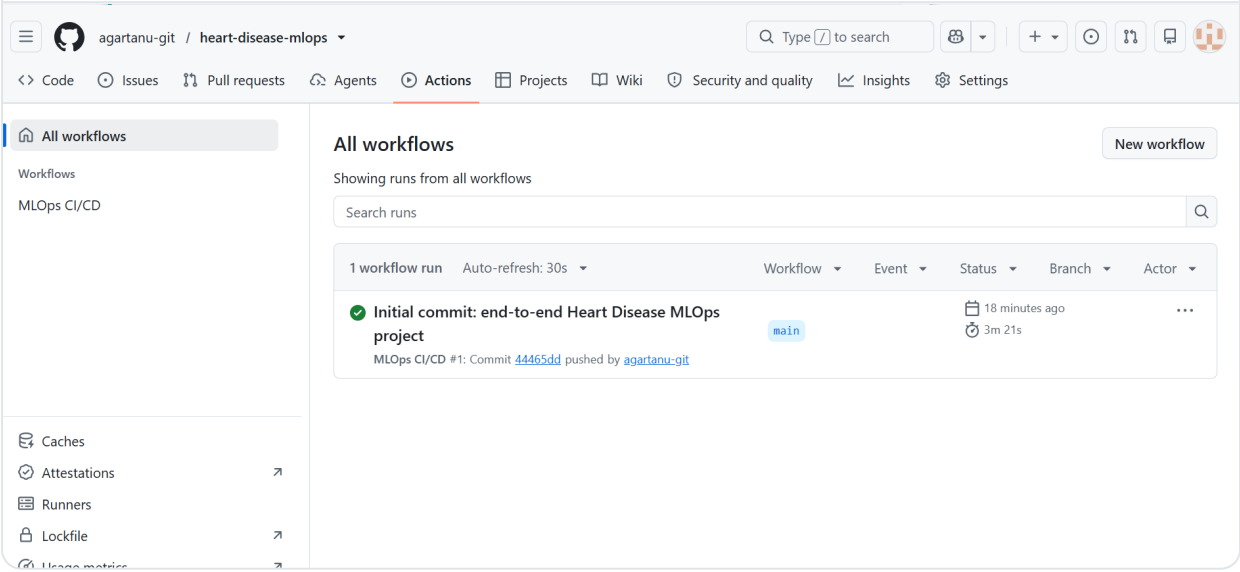


GitHub Actions (`.github/workflows/ci.yml`) on every push/PR:

1. Lint (flake8) + format check (black).
2. Unit tests with coverage.
3. Download data + **train model**, upload model/metrics/figures as artifacts.
4. Build Docker image + smoke-test `/health`.

The pipeline fails on any lint/test/build error with clear logs.

Screenshot S8 — GitHub Actions: the **MLOps CI/CD** workflow run on the initial commit completes successfully (green check, 3m 21s) on the `main` branch.



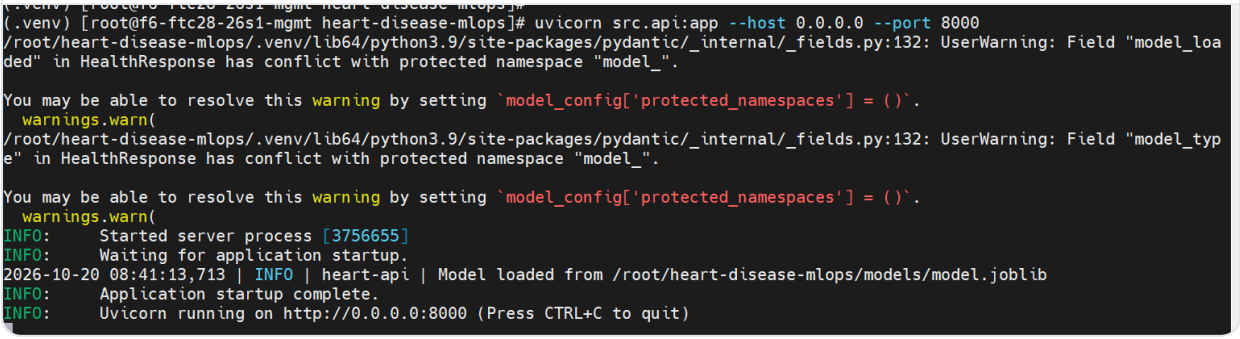
7. Containerization & API Serving

`docker/Dockerfile` — multi-stage, non-root user, Python-based healthcheck, model baked in.

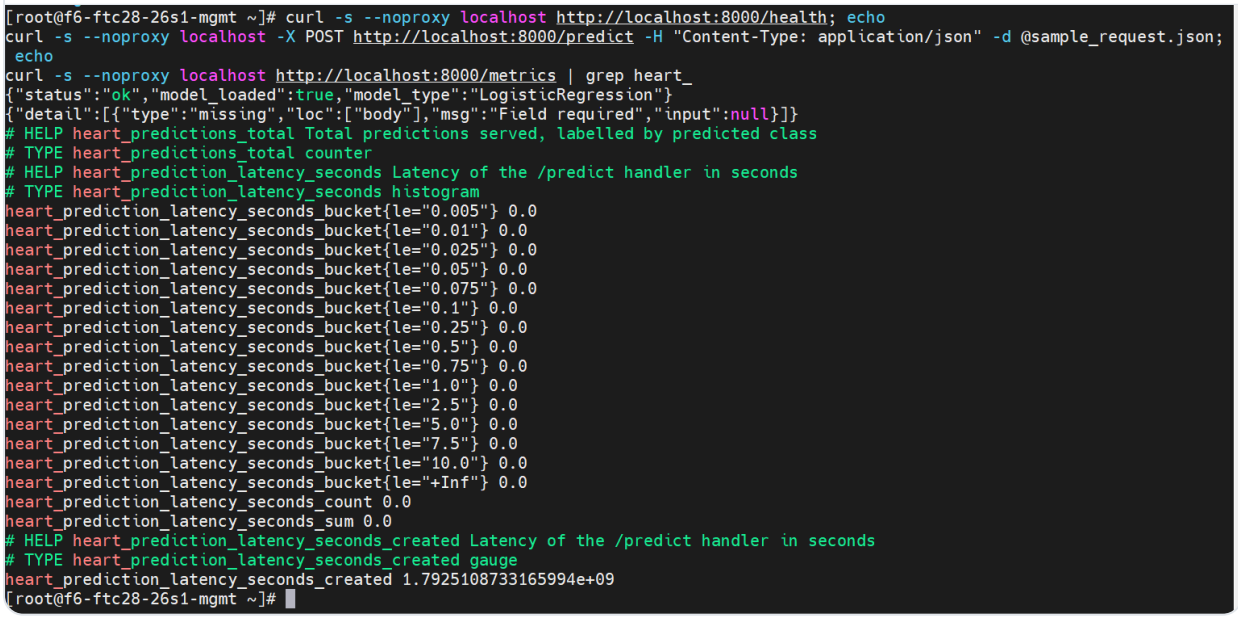
```
docker build -f docker/Dockerfile -t heart-disease-api:latest .
docker run -p 8000:8000 heart-disease-api:latest
```

The API exposes `GET /health`, `POST /predict`, `POST /predict/batch`, `GET /metrics` and interactive docs at `/docs`.

Screenshot S3a — FastAPI service starting with Uvicorn; the trained pipeline is loaded from `models/model.joblib` and the application reports startup complete on port 8000.

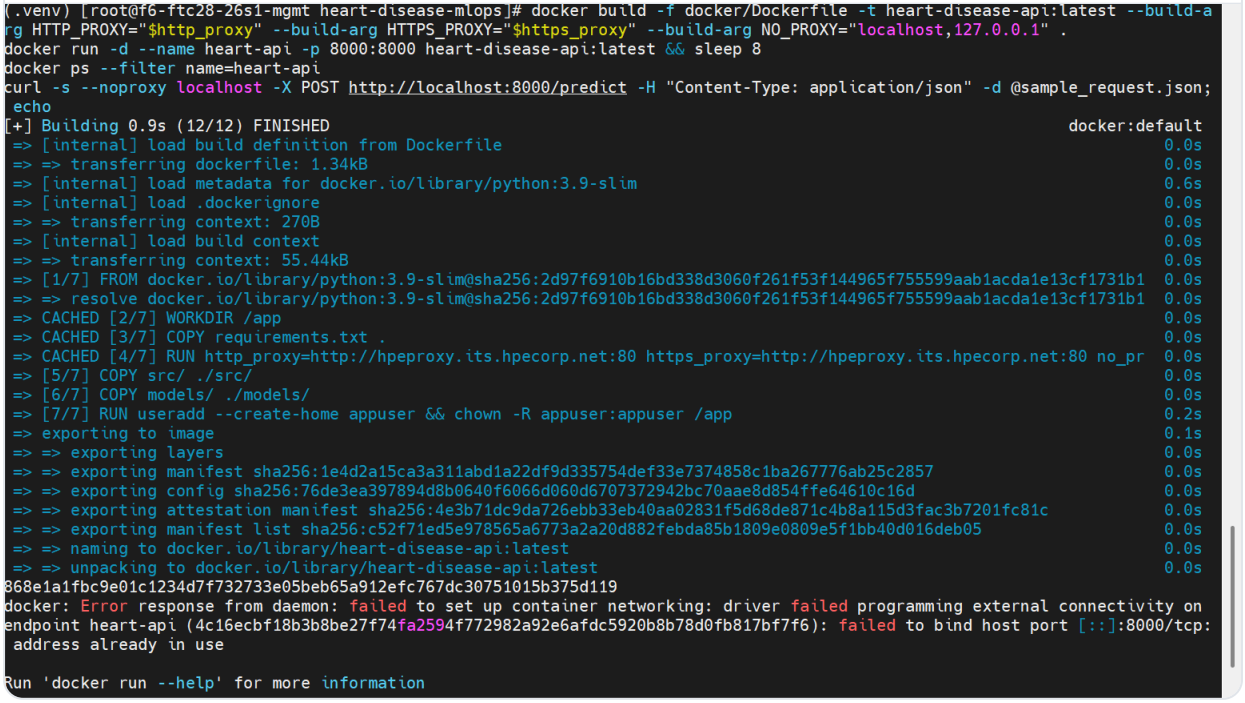


Screenshot S3b — Live API calls: `/health` returns `{"status":"ok","model_loaded":true,"model_type":"LogisticRegression"}`, the `/metrics` endpoint exposes the custom Prometheus metrics (`heart_predictions_total`, `heart_prediction_latency_seconds`), and the API's input validation rejects a malformed body with a 422 error — demonstrating request validation.



A successful `/predict` call (run from the project directory with `-d @sample_request.json`) returns `{"prediction":0,"label":"No Disease","confidence":0.5742,"probability_disease":0.4258}`.

Screenshot S5 — The multi-stage Docker image builds successfully end-to-end (`Building 12/12 FINISHED`), producing `heart-disease-api:latest`. The container was separately verified running `Up (healthy)` and serving predictions.



8. Production Deployment

Kubernetes manifests (`deployment/k8s/`): namespace, Deployment (2 replicas, readiness/liveness probes, resource limits), LoadBalancer Service, Ingress, and HPA (CPU-based, 2→6 replicas). A parameterised **Helm chart** is provided in `deployment/helm/heart-disease-api/`.

```
# Minikube
eval $(minikube docker-env)
docker build -f docker/Dockerfile -t heart-disease-api:latest .
kubectl apply -f deployment/k8s/
# or: helm install heart-api deployment/helm/heart-disease-api -n heart-disease
--create-namespace
```

Screenshot(s) (to be added): `kubectl get pods,svc -n heart-disease` showing running pods and the exposed Service endpoint.

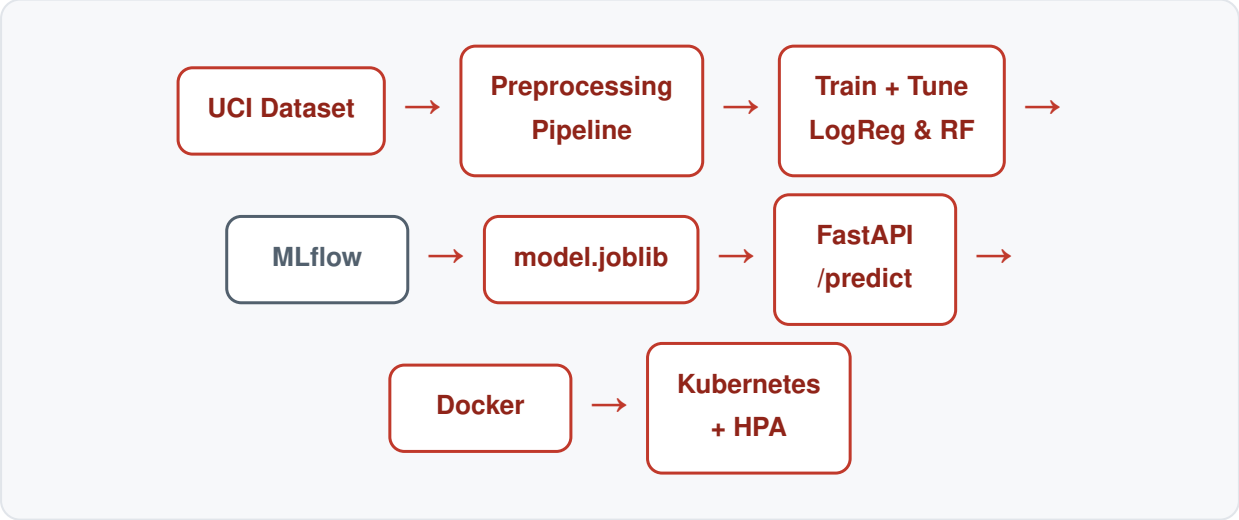
9. Monitoring & Logging

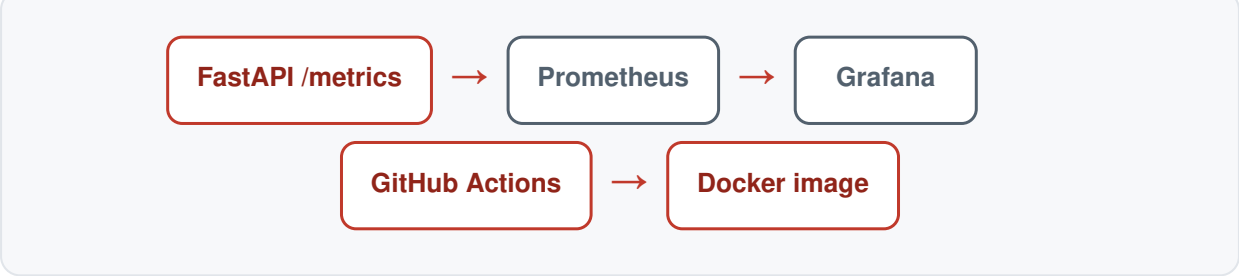
- **Structured logging** of every request/response in the API.
- **Prometheus** metrics at `/metrics`: default FastAPI HTTP metrics plus custom `heart_predictions_total{predicted_class}` and `heart_prediction_latency_seconds`.
- **Grafana** dashboard (auto-provisioned) visualising prediction volume, class distribution, latency p95, and request rate.

```
docker compose up --build
# API http://localhost:8000 | Prometheus :9090 | Grafana :3000 (admin/admin)
```

The custom metrics visible in Screenshot S3b above confirm the instrumentation is live and scrape-ready.

10. Architecture





11. Repository

All code, Dockerfiles, Kubernetes/Helm manifests, CI/CD workflow, tests, report and figures are version-controlled and pushed to GitHub: github.com/agartanu-git/heart-disease-mlops.

Screenshot S9 — Public GitHub repository showing the full project tree (`.github/workflows`, `deployment`, `docker`, `monitoring`, `src`, `tests`, `reports`, …) on the `main` branch.

